Práctica 2: Programación orientada a objetos. Interfaces, tipos genéricos y excepciones

Resumen

En esta práctica presentamos otro algoritmo de aprendizaje automático: *k*-means. Luego añadiremos una interfaz genérica para unificar nuestros algoritmos, y veremos una aplicación práctica de dicha interfaz mediante un sencillo sistema de recomendación. Por último, haremos uso de excepciones para gestionar situaciones anómalas en tiempo de ejecución.

Duración

Para el desarrollo de esta práctica hay previstas **tres** sesiones. A modo orientativo, se indica la duración aproximada para cada parte:

- una sesión para *k*-means,
- media sesión para la interfaz genérica,
- una sesión para el sistema de recomendación, y
- media sesión para el uso de excepciones.

Objetivos

1. Conocer e implementar el algoritmo de agrupamiento *k*-means en una clase.
2. Saber refactorizar tu código para hacer uso de una interfaz genérica.
3. Comprender la utilidad práctica de las interfaces.
4. Saber definir nuevas excepciones y lanzarlas al detectar situaciones anómalas.

1. Agrupamiento (*clustering*) con *k*-means

(duración recomendada: 1 sesión)

Introducción

Considera los datos del fichero `iris.csv` que usamos en la Práctica 1, y supón que **no** disponemos de las etiquetas de cada uno de los ejemplares de flor de Iris que allí aparecen. Nuestro objetivo es agrupar estos datos en tres grupos, uno para cada especie de Iris. (Fíjate en que el número de grupos es un parámetro que suponemos conocido, lo cual no siempre sucede). Para hacer esta agrupación, hemos de utilizar las características disponibles (la longitud y anchura de pétalos y sépalos). ¿Cómo lo podemos hacer?

Existen varias posibilidades, pero todas ellas descansan en la misma idea intuitiva: *La distancia entre los ejemplares del mismo grupo es menor que la distancia entre ejemplares de grupos distintos*, aunque esta afirmación no siempre se cumple para todos los ejemplares.

Esta idea también se puede expresar del siguiente modo: *Si existe un representante de cada grupo, todos los elementos de ese grupo están más cerca del representante de su grupo que de los representantes de los otros grupo*.

La idea anterior es la que usa k-means (https://es.wikipedia.org/wiki/K-medias): encontrar representantes para cada grupo de manera que los ejemplares de cada grupo estén más cerca de su centroide que del centroide de los otros grupos.

Metodología

Entrenamiento

La fase de entrenamiento en *k*-means consiste en encontrar K representantes, uno por cada uno de los grupos que se desee encontrar. El proceso se basa en partir de unos representantes arbitrarios e ir mejorando en un proceso iterativo que implementa el método `train(Table t)`:

```java
public void train(Table datos);
```

De forma detallada, los pasos son:

1. Elegir aleatoriamente un representante por cada grupo, entre los datos de la tabla, sin repetir ninguno.
2. Repetir un número de iteraciones
   a. **Nuevas asignaciones**. Asignar cada dato de la tabla al grupo de cuyo representante está más cerca.
   b. **Nuevos centroides**. Calcular el centroide de cada grupo (según las nuevas asignaciones), que pasará a ser su nuevo representante.

En el primer paso de este algoritmo hacemos una elección aleatoria. Es conveniente fijar la **semilla** de generación de los número aleatorios para que los resultados sean los mismos siempre que utilicemos la misma semilla. La semilla la podemos fijar con `Random random = new Random(long semilla)`.

Puede que algún vídeo (https://www.youtube.com/watch?v=yv2xwnvmbW4) te ayude a comprender el funcionamiento de *k*-means.

> El centroide de un conjunto de puntos es su centro geométrico, que se calcula como el promedio para cada coordenada. Por ejemplo, para puntos con tres dimensiones $e_i(x_i, y_i, z_i)$ el controide $C(x, y, z)$ de M puntos se obtiene como:
>
> $$C(x,y,z) = \frac{\sum_{i=1}^{M} e_i(x_i, y_i, z_i)}{M} = \frac{1}{M}\left(\sum_{i=1}^{M} x_i, \sum_{i=1}^{M} y_i, \sum_{i=1}^{M} z_i\right)$$

El número de grupos o *clusters* (K) es un parámetro de entrada del algoritmo. Por simplicidad, el número de iteraciones (N) también será un parámetro de entrada del algoritmo en lugar de analizar la convergencia (por ejemplo, que los prototipos dejen de cambiar).

Constructor

El constructor de la clase `KMeans` será así:

```java
public KMeans(int numClusters, int numIterations, long seed)
```

donde:

- `numClusters` (K) es el número de grupos,
- `numIterations` (N) es el número de iteraciones, y
- `seed` es la semilla del generador de números aleatorios.

Estimación

El método `estimate()`

```java
public Integer estimate(List<Double> dato)
```

debe devolver un entero en el rango $(1, \ldots, K)$ correspondiente al índice del grupo al que pertenece el dato, es decir, aquel a cuyo representante esté más cerca.

> Fíjate en que el proceso de estimación es equivalente al paso de realizar **nuevas asignaciones** en una de las iteraciones durante el entrenamiento. Aprovecha este hecho para factorizar y reutilizar funciones en `train()` y `estimate()`.

Pruebas

Tal y como lo hemos implementado, el algoritmo *k*-means no es determinista, es decir, varias ejecuciones pueden dar resultados distintos (si la semilla usada para la generación de números aleatorios no es la misma). Esto se debe a que inicializamos los centroides al azar. Existen opciones deterministas, pero no las vamos a considerar.

Pruebas unitarias

En el Aula Virtual, puedes encontrar pruebas, usando JUnit, para verificar varios aspectos de tu implementación. Todas las pruebas deberían pasar.

Además de probar el correcto funcionamiento de vuestro código, las pruebas también pueden servir como pistas para vuestra implementación: métodos y parámetros necesarios, visibilidad de métodos/atributos, etc.

Si las pruebas no se ejecutan porque faltan clases/métodos/atributos correspondientes o no son accesibles, debéis actualizar vuestra implementación para corregir el problema.

Si bien las pruebas prueban ciertos aspectos del código, debido a la naturaleza no determinista del algoritmo antes mencionada, es muy difícil probar exhaustivamente el código, y debemos tener en cuenta que aunque las pruebas pasen, aún podemos tener errores lógicos en el código.

Comprobaciones visuales (OPCIONAL)

De forma complementaria a las pruebas unitarias con JUnit, puedes realizar comprobaciones visuales, menos automáticas, pero muy útiles en el campo del aprendizaje automático y la ciencia de datos. Para ello, te proponemos:

1. Utilizar datos de ejercicios de agrupamiento disponibles en Kaggle (https://www.kaggle.com/datasets/joonasyoon/clustering-exercises).
2. Guardar los resultados del agrupamiento de dichos datos en ficheros CSV (en el aula virtual, proporcionamos una función `saveTable()` como ejemplo - dependiendo de tu código base, es posible que necesites actualizarla).
3. Utilizar la herramienta CSV Plot (https://www.csvplot.com) para visualizar los datos.

En el AV se facilita el código y los datos necesarios para estas pruebas. No necesitas acceder a los datos del enlace anterior.

Puedes aprovechar para comparar qué efecto tiene en el resultado diferentes números de grupos K y de iteraciones N.

2. Refactorización mediante una interfaz genérica

(duración recomendada: máximo de media sesión)

Fíjate en los métodos que tiene nuestra implementación de *k*-Means:

```java
public void train(Table data);
public Integer estimate(List<Double> dato);
```

y los que tenía el algoritmo *k*-NN de la primera práctica:

```java
public void train(TableWithLabels data);
public Integer estimate(List<Double> dato);
```

Como sabes, el cometido de los siguientes métodos es el mismo para ambas clases:

- `train()` construye el modelo a partir de un conjunto de datos y
- `estimate()` da una estimación para un dato nuevo.

A pesar de este aspecto en común, hay algunas diferencias en los tipos. A `train()`, le pasamos como argumento una `Table` o una `TableWithLabels` según el caso, *k*-Means o *k*-NN, respectivamente. Pero recordemos que entre ambas clases existe una relación de herencia (`TableWithLabels` deriva de `Table`).

Entonces:

- Ya que el cometido de los métodos con el mismo nombre en clases distintas es el mismo, vamos a intentar unificarlos en una *interface*.
- Para poder trabajar con distintos tipos, tanto en los argumentos como en el valor de retorno, vamos a trabajar con tipos genéricos.

Metodología

1. Crea una interfaz `Algorithm` con dos métodos:
   - `train()` que use tipos genéricos para el parámetro, y que además garantice que este es `Table` o una clase derivada.
   - `estimate()` con un tipo genérico para su parámetro, y otro tipo genérico para el valor de retorno. Nota: aunque `KMeans` y `KNN` comparten estos tipos, hacemos la interfaz un poco más general para acomodar otros posibles algoritmos de ML donde no es así.
2. Modifica la clase `KMeans` para que implemente la interfaz `Algorithm` asignando a los tipos genéricos los tipos adecuados en la definición de la clase.
3. Idem para la clase `KNN`.

> *Refactorizar* significa mejorar el código sin cambiar su funcionalidad. Por ejemplo, si un método realiza más de una tarea, podemos dividirlo en varios métodos pequeños cada uno de los cuales realiza una tarea única del método original. A su vez, esta descomposición aumenta las posibilidades de reutilización de los nuevos métodos en otras situaciones.

Pruebas

Podemos utilizar la misma batería de pruebas unitarias que para kMeans, ya que la introducción de la interfaz no debe «romper» tu código. Además, en la siguiente parte - sistema de recomendación - incluiremos detalles en algunas pruebas que también prueben la interfaz.

3. Aplicación práctica de la interfaz *Algorithm*: un sistema de recomendación.

(duración recomendada: mínimo de 1 sesión)

Para ilustrar una posible utilidad de la interfaz *Algorithm* planteamos un sencillo sistema de recomendación de elementos (lo que sea: películas, canciones, cursos...). Con este fin, vamos a crear la clase `RecSys` con el constructor y métodos públicos siguientes:

```java
public RecSys(Algorithm algorithm);
public void train(Table trainData);
public void initialise(Table testData, List<String> testItemNames);
public List<String> recommend(String nameLikedItem, int numRecommendations);
```

La idea general es usar los datos `trainData` proporcionados a `train()`, para luego estimar la clase de los datos `testData` proporcionados a `initialise()`. Posteriormente, estas estimaciones serán la base para generar las recomendaciones a partir de un elemento `nameLikedItem` proporcionado a `recommend()`.

Veámos ahora cada uno de estos métodos con más detalle.

`RecSys(Algorithm algorithm)`

Fíjate en que el parámetro del constructor es de tipo `Algorithm` (recuerda, una interfaz genérica) y, al instanciar `RecSys`, le pasaremos al constructor una instancia de un algoritmo concreto, es decir, un objeto de la clase `KMeans` o uno de la clase `KNN`. Esta es la **idea central**. Así, cuando en (métodos de) `RecSys` se hace un `train()` o `estimate()`, se estarán usando los correspondientes métodos de `KMeans` o de `KNN` (`delegación`), según corresponda. De hecho, podríamos pasar cualquier algoritmo de aprendizaje automático que implemente la interfaz `Algorithm` (es decir, podría ser otro algoritmo de agrupamiento (como `KMeans`) u otro algoritmo clasificador (como `KNN`)).

`void train(Table trainData)`

Este método equivaldría a entrenar el sistema de recomendación y se basará en entrenar el algoritmo correspondiente (ya sabes: cualquier algoritmo de aprendizaje automático que implemente la interfaz `Algorithm`; en nuestro caso `KMeans` o `KNN`) usando los datos de entrenamiento proporcionados, `trainData`.

`void initialise(Table testData, List<String> testItemNames)`

Este método equivaldría a arrancar el sistema de recomendación con unos datos distintos a los usados para su entrenamiento. Esta distinción entre conjunto de entrenamiento y prueba es en la práctica muy habitual. Le pasamos también los nombres de los elementos correspondientes a los datos de la tabla (`testData`), con los que trabajará el sistema de recomendación. La cadena *i*-ésima de la lista `testItemNames` se corresponde con el dato *i*-ésimo en la tabla `testData`. El principal cometido de este método es predecir (estimar) la clase (o grupo, según corresponda) de todos los elementos de la tabla `testData`. En otras palabras: debe invocarse al método `estimate()` (de Algorithm) para cada una de las filas de `testData`, y guardarnos estas estimaciones para el momento de hacer recomendaciones.

`List<String> recommend(String nameLikedItem, int numRecommendations)`

Aquí se implementa el algoritmo de recomendación con toda la información disponible. El método recibe el nombre de un elemento que (supuestamente) ha gustado al usuario (`nameLikedItem`), y cuántos elementos queremos que nos recomiende (`clase`). El nombre del elemento debe coincidir exactamente con uno de la lista `testItemNames` proporcionada en la llamada a `initialise()`.

Entonces, el método nos recomienda `numRecommendations` de recomendaciones, originario de `testItemNames`. La idea es que los elementos recomendados tengan la misma *clase* estimada que el elemento gustado.

¡Evidentemente, además de los métodos anteriores, puedes implementar todos los atributos y métodos auxiliares que necesites!

> Cuidado, asegúrate de que el propio elemento no se incluye en la selección de elementos recomendados.
>
> Por otro lado, observa que el número de recomendaciones puede verse como un máximo, porque podría haber menos datos que tengan la etiqueta de clase que nos interesa.

> **Importante**: Hemos simplificado el problema para ajustarlo a las características de la asignatura. Aunque sigue siendo ilustrativo, ten en cuenta que los sistemas de recomendación no funcionan exactamente así. Por ejemplo, como sabes, suelen "cruzar" las preferencias de diferentes usuarios del sistema.

Recomendación de canciones

Aplicaremos `RecSys` a recomendación de canciones. Para ello usaremos una selección de ficheros sobre canciones de Spotify en Kaggle (https://www.kaggle.com/datasets/mrmorj/dataset-of-songs-in-spotify?resource=download) que incluyen atributos de audio (*danceability*, *loudness*...) con los que se entrenan *k*-NN y *k*-Means. Para *k*-NN usaremos, además, el género ("Dark trap", "HipHop"...) de la canción como etiqueta de clase. Se proporciona la clase `SongRecSys` que actúa demo de `RecSys`. Fíjate en que `SongRecSys` usa una interfaz basada en consola para mostrar los resultados de la recomendación. En la última práctica se propondrá una interfaz gráfica equivalente a `SongRecSys` y, ahí viene lo importante, sin tener que modificar `RecSys`, o esa es la idea al menos.

En el AV están disponibles todos los ficheros que necesitamos, ya pre-procesados para nuestro propósito. No hay que usar los de Kaggle; así que no te preocupes si no tienes acceso.

> Date cuenta de que `RecSys` está planteada para que sea general y por ello sirve para diferentes tipos de elementos o productos. Solo se necesitan unos nombres de dichos elementos y algunas características para poder usar un algoritmo de aprendizaje automático. La parte de gestión de ficheros de los elementos concretos la maneja solo `SongRecSys`, mientras que `RecSys` solo trabaja con `Algorithm` y `Table`, y es agnóstica de la procedencia o significado de los datos.

Pruebas

En el Aula Virtual, también encuentras las pruebas para el sistema de recomendación.

Como el sistema de recomendación depende parcialmente de kMeans (que es no determinista), las pruebas solo dan un cierto grado de certeza de que nuestro código es correcto, y debemos tener cuidado de no confiar únicamente en nuestras pruebas para verificar la exactitud de las recomendaciones.

4. Definición y uso de excepciones

(duración recomendada: máximo de media sesión)

Incorpora a tu código al menos las siguientes dos excepciones específicas (personalizadas), que describan situaciones anómalas.

La primera excepción se sitúa en la clase `KMeans`, y nos sirve para gestionar la situación en que el número de grupos indicado K es mayor que el número de de datos N (es decir, algunos grupos estarían vacíos, lo que no tendría sentido). Para ello, necesitas crear una clase nueva, nombrada `InvalidClusterNumberException`, para definir dicha excepción

concreta, extendiendo desde la superclase correcta (tú decides cuál). Asegúrate de que la excepción contenga la información necesaria (número (inválido) de clústeres, número total de datos), para que el programador tenga esta información disponible. Finalmente, tienes que lanzar la excepción donde toca.
La segunda excepción se sitúa en la clase `RecSys`, y nos sirve para gestionar la situación en que un elemento que (supuestamente) ha gustado al usuario (`nameLikedItem`), no existe. Otra vez, tienes que crear una clase nueva, esta vez nombrada `LikedItemNotFoundException`, para definir dicha excepción concreta, extendiendo desde la superclase correcta (tú decides cuál). Asegúrate de que la excepción contenga la información necesaria (nombre inválido de ítem). Finalmente, tienes que lanzar la excepción donde toca.

> Puedes encontrar la jerarquía de clases de excepción de Java y algunas explicaciones aquí: https://rollbar.com/blog/java-exceptions-hierarchy-explained/#.

Pruebas

Las pruebas en Aula Virtual incluyen pruebas para verificar si las excepciones se lanzan correctamente.